

TRABAJO DE LABORATORIO

EMULACIÓN DE UN ATACA LOG4SHELL

Gustavo Landaeta Baeza

Universidad Mayor

Análisis y Explotación de Vulnerabilidades

Prof. Tomas Vera

Septiembre 09, 2024

Índice

Introducción	1
Emulación de un Ataque Log4Shell	2
Objetivo	2
Descripción del Entorno	3
Maquina Virtual	3
Herramientas	3
Aspectos y Comandos Claves	5
Levantar Aplicación Vulnerable-app	6
Copia de Repositorio	6
Construcción de la Aplicación	6
Despliegue de la Aplicación	7
Conclusión	8
Referencias	9

Índice de figuras

1.	Captura de sistema operativo Maquina Virtual Pruebas	3
2.	Captura para verificar si tenemos docker instalado	4
3.	Captura para verificar la versión de docker	4
4.	Captura para verificar la versión de git	5
5.	Vista de contenido de repositorio de Vulnerable-app	6
6.	Vista de la imagen creada localmente en docker de la aplicación Vulnerable-app . . .	6
7.	Ingreso de Comando Despliegue aplicación	7

Índice de tablas

Introducción

En este siglo XXI y en el futuro la seguridad y el análisis de los sistemas se ha vuelto un pilar fundamental para el desarrollo de sistemas confiables, con el crecimiento exponencial de ataques a nivel informático y pérdidas a pequeña y gran escala de entidades y organizaciones públicas y privadas.

Emulación de un Ataque Log4Shell

Objetivo

El objetivo de este informe es realizar por medio de pruebas empíricas un laboratorio emulando un ataque de Log4Shell, desarrollando que el estudiante comprenda las herramientas y técnicas utilizadas en el proceso de la implementación de este ataque. Este ejercicio práctico tiene como finalidad profundizar el entendimiento de las vulnerabilidades y de seguridad de aplicaciones web y fomentar el desarrollo de habilidades técnicas para la identificación y mitigación de riesgos de ciberseguridad.

Descripción del Entorno

Maquina Virtual

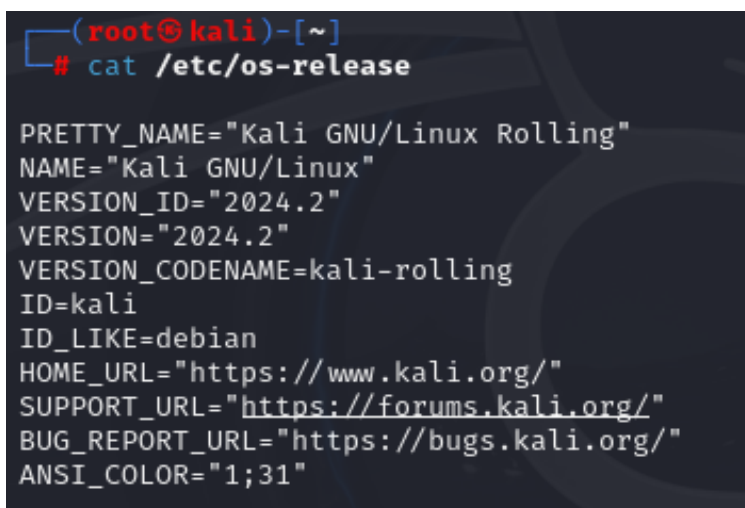
Se descarga maquina virtual de Kali Linux 64 bits desde pagina oficial de Kali Linux <https://www.kali.org/get-kali/#kali-virtual-machines> (*Kali Linux | Penetration Testing and Ethical Hacking Linux Distribution*, 2024) Una vez instalado montamos la imagen de Kali en una maquina virtual Oracle VM versión 7.0.18.

El detalle de la maquina virtual sera el siguiente:

- Sistema operativo: Kali Linux 2024.2
- Memoria Ram: 6gb.
- Procesador: Intel Xeon E5-2680 utilizando 4 procesadores.
- Modo de Red: Nat

Figura 1

Captura de sistema operativo Maquina Virtual Pruebas

A terminal window with a dark background. The prompt is '(root@kali)-[~]'. The command '# cat /etc/os-release' has been executed. The output shows the following system information: PRETTY_NAME='Kali GNU/Linux Rolling', NAME='Kali GNU/Linux', VERSION_ID='2024.2', VERSION='2024.2', VERSION_CODENAME=kali-rolling, ID=kali, ID_LIKE=debian, HOME_URL='https://www.kali.org/', SUPPORT_URL='https://forums.kali.org/', BUG_REPORT_URL='https://bugs.kali.org/', and ANSI_COLOR='1;31'.

```
(root@kali)-[~]  
# cat /etc/os-release  
  
PRETTY_NAME="Kali GNU/Linux Rolling"  
NAME="Kali GNU/Linux"  
VERSION_ID="2024.2"  
VERSION="2024.2"  
VERSION_CODENAME=kali-rolling  
ID=kali  
ID_LIKE=debian  
HOME_URL="https://www.kali.org/"  
SUPPORT_URL="https://forums.kali.org/"  
BUG_REPORT_URL="https://bugs.kali.org/"  
ANSI_COLOR="1;31"
```

Herramientas

Las herramientas que utilizaremos en la maquina virtual seran Docker y Git.

- Docker: Se utilizara docker para crear las imágenes de todo el contenido necesario para el uso de la aplicación atacada y aplicación atacante. Estas imágenes luego se deben iniciar en un contenedor y exponer en puertos.
- GIT: lo utilizaremos para verificar el control de version del repositorio donde se descargarán los archivos necesarios para las pruebas.

El primer paso es verificar si la versión de Kali a utilizar contiene estas herramientas. En caso de no tenerlas se deben instalar en el ambiente de laboratorio.

Se verifica si docker esta instalado por medio del comando **docker --version**. Este comando verifica la version de docker instalada en el equipo Figura 2

Figura 2

Captura para verificar si tenemos docker instalado

```
(root@kali)-[~]  
# docker --version  
Command 'docker' not found, but can be installed with:  
apt install docker.io  
apt install podman-docker
```

Al no tener docker instalado se procede a instalar docker de forma sencilla ya que kali al reconocer que no tiene la aplicación proporciona el comando para instalarlo dentro de su herramienta gestora de paquetes apt (Advance Package Tool)(*Package management / Ubuntu*, s.f.). Luego de ingresar el comando **apt install docker.io** podemos nuevamente verificar que tenemos la version instalada de docker

Figura 3

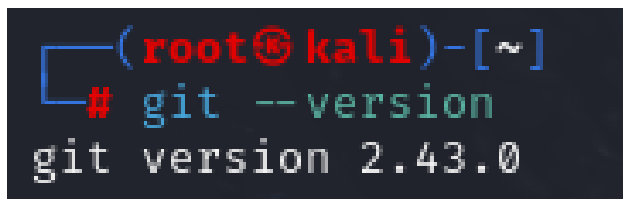
Captura para verificar la versión de docker

```
(root@kali)-[~]  
# docker --version  
Docker version 20.10.25+dfsg1, build b82b9f3
```

Ahora procederemos a verificar la versión de Git con el comando `git --version`. Donde podemos verificar que la versión de Kali si tiene ya instalada una versión de git.

Figura 4

Captura para verificar la versión de git



```
(root@kali)-[~]  
# git --version  
git version 2.43.0
```

Aspectos y Comandos Claves

Para un desarrollador y un usuario de Docker y Git existen varios aspectos claves dependiendo la necesidad del trabajo que se este realizando, para este laboratorio los comandos claves a utilizar serán los siguientes:

- `docker exec -it my_container {comando de ruta u otro}`: Con este comando podemos ingresar de modo interactivo a la terminal del contenedor donde podemos colocar alguna ruta en especial. Las más utilizadas son `/bin/bash`, `/bin/sh` y `ls`.
- `docker inspect`: Con este comando podemos inspeccionar todo lo que contiene el contenedor de Docker. Para este laboratorio se utilizará un comando un poco más especial para obtener la IP de cada contenedor inicializado. Este comando será el siguiente: `docker inspect -f '{{range.NetworkSettings.Networks}}{{.IPAddress}}{{end}}' {nombre de contenedor o ID}`.
- `docker ps`: Para verificar si el contenedor está en ejecución.
- `docker images`: Para verificar las imágenes contenidas localmente en Docker.
- `git clone {url repositorio}`: Para copiar localmente el o los repositorios que necesitemos.

Levantar Aplicación Vulnerable-app

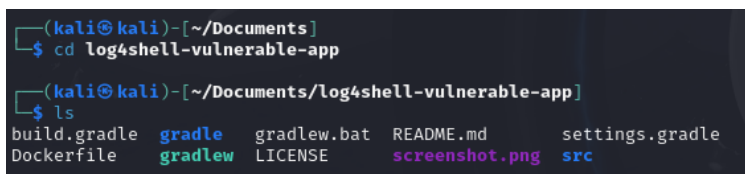
Para poder levantar la aplicación seguiremos una serie de pasos para poder desplegar la aplicación que contiene la vulnerabilidad.

Copia de Repositorio

Procederemos a copiar de forma local desde el repositorio de la aplicación Vulnerable-app, para esto necesitamos ejecutar el comando **git clone https://github.com/darryk10/log4shell-vulnerable-app.git**. Con este comando copiaremos de forma local el repositorio de la aplicación que contiene la vulnerabilidad.

Figura 5

Vista de contenido de repositorio de Vulnerable-app



```
(kali@kali)-[~/Documents]
$ cd log4shell-vulnerable-app

(kali@kali)-[~/Documents/log4shell-vulnerable-app]
$ ls
build.gradle  gradle  gradlew.bat  README.md  settings.gradle
Dockerfile   gradlew LICENSE  screenshot.png  src
```

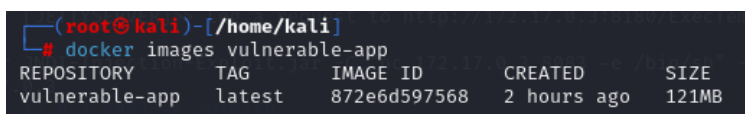
Construcción de la Aplicación

Una vez descargado el repositorio debemos entrar a la carpeta del repositorio descargado y con el comando **ls** revisar el contenido del repositorio.

Desde la pagina del repositorio podemos verificar los pasos recomendados. En primer lugar recomiendan construir la imagen de la aplicación por medio del comando **docker build . -t vulnerable-app**. Se debe tener en cuenta que para construir la imagen debo tener un archivo receta Dockerfile dentro de la carpeta donde ejecutare el comando.

Figura 6

Vista de la imagen creada localmente en docker de la aplicación Vulnerable-app



```
(root@kali)-[/home/kali]
# docker images vulnerable-app
```

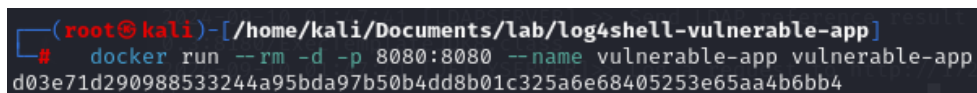
REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
vulnerable-app	latest	872e6d597568	2 hours ago	121MB

Despliegue de la Aplicación

Para el despliegue de la aplicación ejecutaremos el comando **docker run -rm -d -p 8080:8080 --name vulnerable-app vulnerable-app**. Una vez ejecutado el comando se expondrá la aplicación en el puerto 8080.

Figura 7

Ingreso de Comando Despliegue aplicación



```
(root@kali)-[/home/kali/Documents/lab/log4shell-vulnerable-app]
# docker run --rm -d -p 8080:8080 --name vulnerable-app vulnerable-app
d03e71d290988533244a95bda97b50b4dd8b01c325a6e68405253e65aa4b6bb4
```

Nos podemos percatar en la Figura 7 que al iniciar la aplicación docker nos proporciona un ID del contenedor el cual nos servirá para verificar algunos aspectos claves para poder trabajar con nuestro contenedor desplegado.

Conclusión

En conclusión, el papel que juega el DPO es fundamental en el mundo del siglo XXI, ya que asegura que las organizaciones no solo cumplan con las obligaciones legales, sino que también adopten un enfoque dinámico y responsable hacia la protección de datos. A medida que la tecnología y las regulaciones avanzan, la importancia del DPO continuará creciendo, consolidándose como un pilar fundamental en la gestión de la privacidad y la seguridad de la información.

Referencias

Kali Linux / Penetration Testing and Ethical Hacking Linux Distribution. (2024).

<https://www.kali.org/>.

Package management / Ubuntu. (s.f.). Descargado de <https://ubuntu.com/server/docs/package>

`-management`